

Operators & Types of Operators

Operators are special symbols in **Python** that carry out arithmetic or logical computation. The value that the **operator** operates on is called the operand. Python language supports the following types of operators.

1. **Arithmetic Operators**
2. **Comparison (Relational) Operators**
3. **Assignment Operators**
4. **Logical Operators**
5. **Bitwise Operators**
6. **Membership Operators**
7. **Identity Operators**

Types of Operators

1. Arithmetic Operators:

Python Arithmetic Operators		
Operator	Name	Example
+	Addition	$a + b$
-	Subtraction	$a - b$
*	Multiplication	$a * b$
/	Division	a / b
%	Modulus	$a \% b$
**	Exponentiation	$a ** b$
//	Floor division	$a // b$

```
#Arithmetic operators
```

```
a = 50
```

```
b = 15
```

```
I
```

```
print("Addition = ",a+b)
```

```
print("Subtraction = ",a-b)
```

```
print("Multiplication = %d" %(a*b))
```

```
print("Division = {}".format(a/b))
```

```
print(a%b)
```

```
print(a//b)
```

```
print(a**2)
```

Types of Operators

2. Relational (Comparison) Operators:

Relational operators are symbols that perform operations on data and return a result as true or false depending on the comparison conditions.

Relational Operators are those that discover relationship between the two operands provided to them. Relational operators or comparison operators are used to compare between values.

Types of Operators

2. Relational (Comparison) Operators:

Operator	Description
<	Less than
>	Greater than
<=	Less than or equal to
>=	Greater than or equal to
==	Equal to
!=	Not equal to

```
#Relational operators
print(a<b)
print(a>b)
print(a>=b)
print(a==b)
print(a!=b)
```

Types of Operators

3. Assignment Operators :

Assignment Operators are those that help in assigning a value to the variable.

`a = 5` is a simple assignment operator that assigns the value 5 on the right to the variable a on the left.

There are various compound operators in Python like `a += 5` that adds to the variable and later assigns the same. It is equivalent to `a = a + 5`.

Types of Operators

3. Assignment Operators :

Operators	Description
=	Assignment
/=	Divide and Assign
+=	Add and assign
-=	Subtract and Assign
*=	Multiply and assign
%=	Modulus and assign
**=	Exponent and assign
//=	Floor division and assign

```
#Assignment operators
x = 100
print(x)
x/=10
print(x)
x*=10
print(x)
```


Types of Operators

4. Bitwise Operators :

Bitwise operator works on bits and performs bit by bit operation. Assume if $a = 60$; and $b = 13$; Now in binary format they will be as follows:

$a = 0011\ 1100$

$b = 0000\ 1101$

$a \& b = 0000\ 1100$

$a | b = 0011\ 1101$

$a \wedge b = 0011\ 0001$

$\sim a = 1100\ 0011$

4. Bitwise Operators :

OPERATOR	DESCRIPTION	SYNTAX
&	Bitwise AND	$x \& y$
	Bitwise OR	$x y$
~	Bitwise NOT	$\sim x$
^	Bitwise XOR	$x \wedge y$
>>	Bitwise right shift	$x >>$
<<	Bitwise left shift	$x <<$

```
#Bitwise operators
A = 60
B = 13
print(A&B)
print(A|B)
print(A^B) #Bitwise X-OR operator
```

Types of Operators

5. Logical Operators :

Logical Operators are utilised to complete Boolean tasks like AND, OR etc.

Logical Operators are used to perform certain logical operations on values and variables. These are the special reserved keywords that carry out some logical computations. The value the operator operates on is known as Operand. In Python, they are used on conditional statements (either True or False), and as a result, they return Boolean only (True or False). They are used to combine conditional statements

Types of Operators

5. Logical Operators :

There are following logical operators supported by Python language:

Logical AND

Logical OR

Logical NOT

```
#Logical operator  
L = 10  
M = 20
```

```
if (L>5 and M>15):  
    print("Yes")  
else:  
    print("No")
```

```
#Membership operator  
List = [10,20,30,40]  
print(20 in List)  
print(200 not in List)
```

Types of Operators

6. Membership Operators :

Python's membership operators test for membership in a sequence, such as strings, lists, or tuples. There are two membership operators

in

not in

in : Evaluates to true if it finds a variable in the specified sequence and false otherwise.

not in : Evaluates to true if it does not find a variable in the specified sequence and false otherwise.

Types of Operators

7. Identity Operators :

Identity operators compare the memory locations of two objects. There are two Identity operators

is : Evaluates to true if the variables on either side of the operator point to the same object and false otherwise.

is not :Evaluates to false if the variables on either side of the operator point to the same object and true otherwise.

```
#Identity Operator  
s=10  
print(s is 10)  
print(s is not 10)
```

Introduction to Decision making

Types of decision making statements

- **if statement**
- **if...else statement**
- **nested if statement**

elif statement

Decision Making

Decision making is a significant perspective in any programming language. It assists with taking an alternate way in the code dependent on the dynamic outcomes. This helps a great deal in process automation kind of stuff, where sooner or later in the process there could be numerous different ways, and the state by then chooses the way to be taken.

Decision Making

Decision structures evaluate multiple expressions which produce TRUE or FALSE as outcome. You need to determine which action to take and which statements to execute if outcome is TRUE or FALSE otherwise.

Python programming language provides following types of decision making statements.

- **if Statements**
- **if...else statements**
- **nested if statements**

Types of Decision Making Statements:

1) if statement :

An **if** statement consists of a Boolean expression followed by one or more statements.

A Boolean expression is evaluated and if TRUE, a block of statements are executed else, the execution flow continues with the execution of statements after if statement.

Types of Decision Making Statements:

1) if statement :

The syntax of if statement is:

if expression:
statement(s)

If the **expression** is evaluated to TRUE, the statement gets executed. But if it's FALSE, nothing happens.

1) if statement :

Example:

```
a = 15
```

```
if (a > 10): #you can also write as if a>10:  
    print("a is greater")
```

Types of Decision Making Statements:

2) If...else Statement :

An **if** statement can be followed by an optional **else** statement, which executes when the Boolean expression is **false**.

A Boolean expression is evaluated and if TRUE, a block of statements are executed else another block of statements are executed, then the execution flow continues with the execution of statements after if statement.

Types of Decision Making Statements:

2) If...else Statement :

The syntax of **if...else** statement is:

```
if expression:  
    statement(s)  
else:  
    statement(s)
```

The else part is optional and is only evaluated if **expression** is evaluated to be FALSE.

Types of Decision Making Statements:

2) If...else Statement :

Example:

x = 305

y = 405

if x > y:

 print("x is greater")

else:

 print("y is greater")

```
a = 150
if a>1000:    #you can also write as if a>10:
    print("a is greater")
```

```
x = 505
y = 405
if x > y:
    print("x is greater")
else:
    print("y is greater")
```

```
p=50
if p%2==0:
    print("Even number")
else:
    print("Odd number")
```

Types of Decision Making Statements:

3) Nested if statements:

A **nested if** is an **if statement** that is the target of another if statement. Nested if statements means an **if statement** inside another **if statement**. Yes, Python allows us to nest if statements within if statements. i.e., we can place an **if statement** inside another **if statement**.

Types of Decision Making Statements:

3) Nested if statements:

Example:

```
a=56
if(a > 40):
    if(a > 45):
        print("a is greater than 35")
    else:
        print("a is less than 45")
else:
    print("a is less than 40")
```


Decision Making

elif statement:

The **elif** statement allows you to check multiple expressions for TRUE and execute a block of code as soon as one of the conditions evaluates to TRUE. Similar to the **else**, the **elif** statement is optional. However, unlike **else**, for which there can be at most one statement, there can be an arbitrary number of **elif** statements following an **if**.

elif - is a keyword used in Python replacement of else if to place another condition in the program. This is called chained conditional.

Decision Making

elif statement:

The syntax of **elif** statement is:

```
if expression1:  
    statement(s)  
elif expression2:  
    statement(s)  
elif expression3:  
    statement(s)  
else:  
    statement(s)
```

Introduction to Loops

Types of Loops

- **for loop**
- **while loop**
- **nested loop**

Loop Control Statements

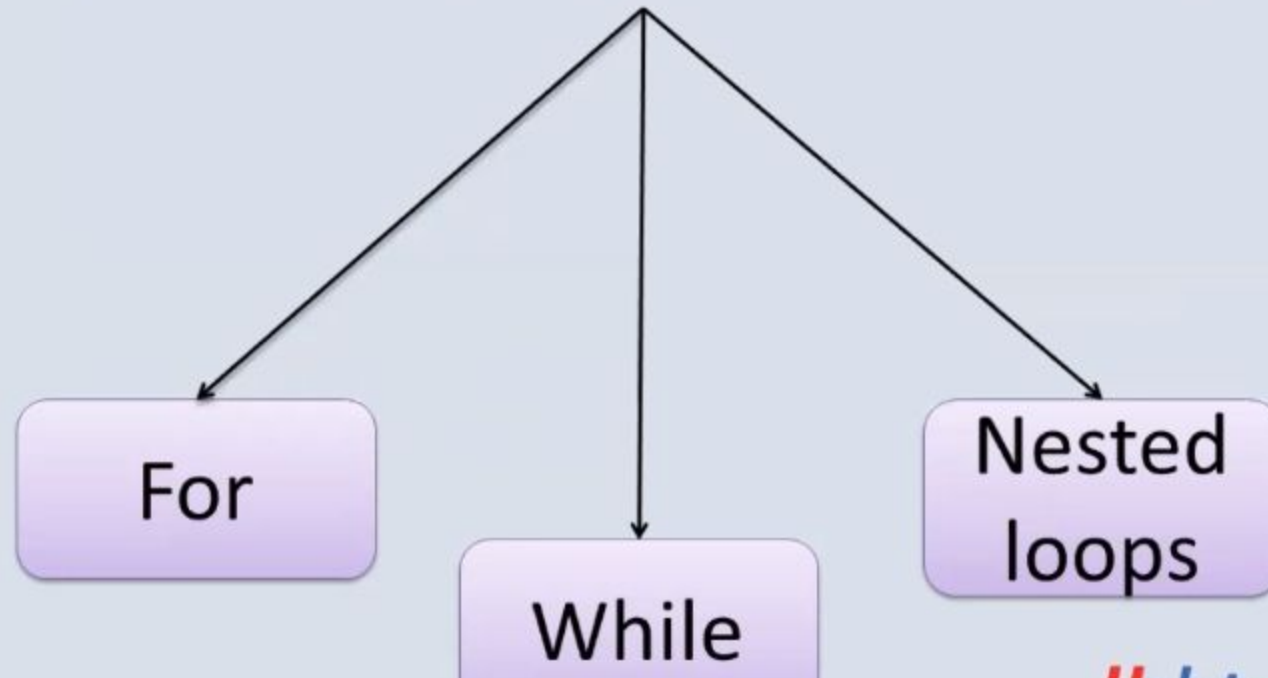
- **Break, continue and pass statement**

Introduction to Loops

There could be a situation where you got to execute a block of code several numbers of times. Generally, statements are executed sequentially. The primary statement during a function is executed first, followed by the second, and so on.

A loop statement allows us to execute a statement or group of statements multiple times.

Types of Loops



Types of Loops

For loop:

- Executes a sequence of statements multiple times and abbreviates the code that manages the loop variable.
- A for loop is used for iterating over a sequence.
- It is like an iterator method as found in other object-orientated programming languages.



Types of Loops

For loop:

Syntax:

```
for var in sequence:  
    statements(s)
```

Here **var** is a variable that is used for iterating over a **sequence**. On every iteration it takes the **next value** from **sequence** until the end of sequence is reached.

sequence : A sequence of values that will be assigned to the target variable **var**. Values are provided using a list or a string or from the built-in function `range()`.

Types of Loops

while loop:

- Repeats a statement or group of statements while a given condition is TRUE. It tests the condition before executing the loop body.
- While Loop is used to execute number of statements or body till the condition passed in while is true.
- Once the condition is false, the control will come out of the loop.

Types of Loops

while loop:

Syntax:

while expression:
statement(s)

Here, **statement(s)** may be a single statement or a block of statements. The **condition** may be any **expression**, and true is any non-zero value. The loop iterates while the condition is true.

When the condition becomes false, program control passes to the line immediately following the loop.

Types of Loops

while loop:

Example-1 :

```
count = 0
```

```
while (count < 6):
```

```
    print ('The count is:', count)
```

```
    count = count + 1
```

Example-2:

```
table = 8
```

```
while table < 81 :
```

```
    print (table)
```

```
    table+=8
```

Types of Loops

nested loop:

The placing of one loop inside the body of another loop is named nesting. Once you “nest” two loops, the outer loop takes control of the amount of complete repetitions of the inner loop. Thus inner loop is executed N- times for each execution of outer loop.

Python programming language allows to use one loop inside another loop .

Types of Loops

nested loop:

Syntax for nested for loop :

```
for iterating_var in sequence:  
    for iterating_var in sequence:  
        statements(s)  
statements(s)
```

Syntax for nested while loop :

```
while expression:  
    while expression:  
        statement(s)  
statement(s)
```

Types of Loops

nested loop:

Example:

```
for i in range(1,6):  
    for j in range (1,i+1):  
        print (i, end=" ")  
    print ()
```


Types of Loops

Infinite loop:

A loop becomes infinite loop if a condition never becomes FALSE. You must use caution when using while loops because of the possibility that this condition never resolves to a FALSE value. This results in a loop that never ends. Such a loop is called an infinite loop.

An infinite loop might be useful in client/server programming where the server needs to run continuously so that client programs can communicate with it as and when required.

Types of Loops

Infinite loop:

Example:

```
cnt = 10
while (cnt > 9):
    print ('The count is:', cnt)
```

Loop Control Statements

Loop control statements change execution from its normal sequence. When execution leaves a scope, all automatic objects that were created in that scope are destroyed. Python supports the following control statements.

break

continue

pass

Loop Control Statements

break statement:

Terminates the loop statement and transfers execution to the statement immediately following the loop.

The most common use for **break** is when some external condition is triggered requiring a hasty exit from a loop. The **break** statement can be used in both *while* and *for* loops.

```
for i in range(11,200):  
    if(i==20):  
        break  
    print(i)
```

```
for i in range(16,21):  
    if(i==18):  
        continue  
    print(i)
```

```
for i in range(26,41):  
    if(i==37):  
        pass  
    print(i)
```

Loop Control Statements

break statement:

Example:

```
for i in range(11,21):  
    if(i==17):  
        break  
    print(i)
```


Loop Control Statements

continue statement:

Causes the loop to skip the remainder of its body and immediately retest its condition prior to reiterating.

It returns the control to the beginning of the while loop. The **continue** statement rejects all the remaining statements in the current iteration of the loop and moves the control back to the top of the loop.

The **continue** statement can be used in both *while* and *for* loops.

continue statement:

Example:

```
for i in range(16,21):  
    if(i==18):  
        continue  
    print(i)
```


Loop Control Statements

Pass statement:

The pass statement in Python is used when a statement is required syntactically but you do not want any command or code to execute.

The **pass** statement is a *null* operation; nothing happens when it executes. The **pass** is also useful in places where your code will eventually go, but has not been written yet.

Loop Control Statements

Pass statement:

Example:

```
for i in range(26,41):  
    if(i==37):  
        pass  
    print(i)
```